

Lab 5 – Buscar en Wikipedia con Elasticsearch

Big Data

28 de julio de 2026

Hoy vamos a construir un motor de búsqueda para Wikipedia.

Primero, indexaremos la URL y las palabras claves en el título y el resumen (primer párrafo) de todos los artículos en Wikipedia en español usando Elasticsearch (un motor de búsqueda y análisis distribuido que permite la creación de índices invertidos del texto en documentos). Luego implementaremos la función de búsqueda: escribe una palabra clave, recupera los artículos que coinciden.

- Consulta los datos de muestra aquí: <http://aidanhogan.com/teaching/data/wiki/es/es-wiki-articles-1k.tsv>. Contiene las primeras 1000 líneas de los datos, con un artículo en cada línea. Cada línea tiene la URL, el título y el resumen (delimitado por tabulación) de un artículo. Los datos completos (que usaremos más adelante) están disponibles en el servidor y contienen 964,838 líneas en el mismo formato.
- Luego, presentamos Elasticsearch, que en su núcleo ofrece un índice invertido distribuido (junto con otras características para indexar documentos similares a JSON). Se puede interactuar con Elasticsearch a través de API REST basada en HTTP (con los métodos GET, POST, PUT, DELETE, etc.) utilizando solicitudes y respuestas en formato JSON. El comando `curl` utilizado aquí es un comando general para enviar una solicitud a través de HTTP y recibir una respuesta. Inicie sesión en el clúster como de costumbre. La API REST de Elasticsearch está alojada en `cm:9200`.
 - Para ver el estado del cluster: `curl -X GET "cm:9200/_cat/health?v&pretty"`
 - Para agregar un documento al índice “customer” (el índice se creará si no existe; tenga en cuenta que este comando es una sola línea):

```
curl -X PUT "cm:9200/customer/_doc/1?pretty"
-H 'Content-Type: application/json' -d '{ "name": "Fin Shepard" }'
```
 - Para obtener un documento indexado por id: `curl -X GET "cm:9200/customer/_doc/1?pretty"`
 - Para mostrar los índices existentes: `curl "cm:9200/_cat/indices?v"`
 - Para eliminar un índice: `curl -X DELETE "cm:9200/customer?pretty"`
 - Para crear un índice llamado `mywiki` con un solo campo `TITLE` de tipo `text`, con `store` establecido en `true`, y analizador `standard`:

```
curl -X PUT "cm:9200/mywiki?pretty" -H 'Content-Type: application/json' -d'
{
  "mappings" : {
    "_doc" : {
      "properties" : {
        "TITLE" : {
          "type" : "text",
          "store" : "true",
          "analyzer" : "standard"
        }
      }
    }
  }
}
```

Entonces, ¿qué significa ese último comando? En realidad, hay algunos conceptos importantes para aclarar:

- Un *field (campo)* es un atributo de un documento, como el título, URL, última-actualización, resumen, etc. Posteriormente, podemos seleccionar en qué campos del documento queremos buscar, por eso damos nombres a los campos como **TITLE**, para que podamos consultarlos más adelante. El contenido de los campos debe tener un **type (tipo)**; por ejemplo, un campo **TÍTULO** puede contener texto, un campo **ÚLTIMA-ACTUALIZACIÓN** puede contener fechas, etc.
 - A continuación, recuerde que los índices invertidos típicamente dividirán un fragmento de texto, digamos un título o un resumen, en varias palabras. También se pueden normalizar las palabras (Ej., **chileno** → **chile**), perdiendo información sobre el texto original. Por lo tanto, a menudo es imposible recuperar el texto original del índice invertido. Si deseamos poder recuperar el contenido original más tarde, por ejemplo, queremos mostrar el título o el resumen en los resultados de búsqueda, debemos establecer **store** en **true**, que permite almacenar el contenido original exacto (con el costo de un índice más grande). Si bien aún podemos buscar campos con **store** establecido en **false**, no podremos recuperar el contenido original de esos campos.
 - Finalmente, el **analyzer** es lo que analiza las palabras del texto, las normaliza (incluyendo mayúsculas, acentos, derivaciones, etc.), y filtra las palabras vacías. El analizador **standard** funciona genéricamente para todos los idiomas, pero podemos elegir analizadores más específicos, como los de idiomas específicos, como **spanish**. Como un pequeño ejemplo, un analizador estándar no considerará la palabra **ese** como una palabra vacía, mientras que el analizador de español lo hará. Es importante usar el mismo analizador para indexar y consultar; de lo contrario, si se utilizan diferentes análisis y/o normalizaciones, podrían surgir “mismatches” para palabras normalizadas de diferentes maneras. Por esta razón, Elasticsearch configura el analizador a nivel de un índice particular.
- Si bien podríamos usar la API REST para indexar los documentos uno por uno, tendría más sentido construir el índice mediante programación. Para esto, utilizaremos un cliente Java para Elasticsearch (alternativamente, podríamos codificar un cliente para la API REST si lo deseáramos). Descargue el proyecto de código de u-cursos y ábralo en Eclipse o el IDE de Java que prefiera. La primera clase Java con la que trabajaremos indexará el archivo grande en el índice invertido Elasticsearch. La segunda recibirá términos para buscar y usará el índice para encontrar los artículos más relevantes de Wikipedia.
 - En ambas clases, la cadena **@TODO** los guiará a los lugares donde necesita completar el código (hay dos partes para completar en general). También tenga en cuenta que utilizamos una enumeración para asegurarnos de que los nombres de los campos sean coherentes al realizar consultas e indexar; específicamente, usamos:

- `FieldNames.URL.name()` para la url (que es la cadena URL)
- `FieldNames.TITLE.name()` para el título (que es la cadena TITLE)
- `FieldNames.ABSTRACT.name()` para el resumen (que es la cadena ABSTRACT)
- `FieldNames.MODIFIED.name()` para el tiempo (que es la cadena MODIFIED)

El uso de `FieldNames.URL.name()` en lugar de la cadena URL (por ejemplo) hace que sea más fácil cambiar el nombre del campo más adelante, mantener el nombre del campo consistente, evitar errores ortográficos, etc.

- Primero abra `BuildWikiIndexBulk`. El método principal ya está escrito para usted, al igual que parte del código de indexación. Sin embargo, en lugar de indexar solo la URL del documento (`tabs[0]`), también debe indexar el título del documento (`tabs[1]`), el resumen del documento (`tabs[2]`: si está disponible, que puede verificar con `tabs.length > 2`), y la hora a la que se indexó el documento (`System.currentTimeMillis()`). Extienda el código para indexar estos campos.
- Luego abra `SearchWikiIndex`. Nuevamente, el método principal ya está escrito para usted, al igual que algunos de los códigos de búsqueda. Sin embargo, en lugar de buscar solo el título del documento, también debe buscar el resumen (si está disponible). También debe recuperar e imprimir detalles de cada resultado para la salida estándar: su título (ya recuperado), su URL y su resumen. Extienda el código a lo largo de estas líneas.
- Una vez que hayas terminado, ¡es hora de probarlo! Construya el jar con `build.xml`. Copie el jar en su carpeta en el clúster. Ejecute el jar con un nombre de índice único (mejor usar un nombre en **minúscula** para `INDEX-NAME` porque a veces Elasticsearch no reconoce nombres en mayúscula):

```
• java -jar gdd-elastic.jar BuildWikiIndexBulk -i DATA -igz -o INDEX-NAME
```

donde `DATA` está en `/data/uhadoop/shared/wiki/es-wiki-articles.tsv.gz`. Tenga en cuenta que si el índice no se definió previamente, se creará con la configuración predeterminada (incluido el analizador estándar); cualquier campo en documentos indexados también se creará con la configuración predeterminada.

- Una vez finalizada la indexación (debería leer 964,838 líneas), puede ejecutar el jar con:

```
• java -jar gdd-elastic.jar SearchWikiIndex -i INDEX-NAME
```

para buscar en el índice.

- Prueba buscar “trump”. ¿Los resultados tienen sentido? Copie la consulta y los resultados.
- Ejecute otras cuatro búsquedas (con resultados no vacíos) y copie los resultados.
- ENTREGA: tus dos clases – `BuildWikiIndexBulk` y `SearchWikiIndex` – a u-cursos, junto a un archivo de texto con los resultados de las búsquedas “trump” y las otras cuatro búsquedas que elegiste.
- OPCIONAL: Intente variar los factores de impulso boost en `SearchWikiIndex` entre título y resumen y vea cómo afecta los resultados de algunas consultas. ¿Cómo cambian los resultados?